Studio Nx

Backend(FastAPI/Python) 28 — decode_refresh_token()

2026-10-12

Backend(FastAPI/Python) 28 — `decode_refresh_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인: Refresh Token 검사 (PRD 01 FR-105)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	27 `create_refresh_token()` · 29 `authenticate()`

1. 하는 일

리프레시 쿠키에 담긴 토큰이 **진짜이고 아직 유효한 Refresh Token**인지 확인하고, 회원 번호를 돌려줍니다. 실패하면 401로 응답합니다.

`decode_refresh_token("eyJ…")` → `7`

05번 `decode_access_token()`과 같은 검사이며, `type`만 `"refresh"`여야 한다는 점이 다릅니다.

함수	받아들이는 토큰
05 `decode_access_token()`	`type: "access"`만 받습니다
28 `decode_refresh_token()`	`type: "refresh"`만 받습니다

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/refresh` (33)	모든 페이지의 `api.js` `refreshAccessToken()` (JS 01)	쿠키 `snx_refresh`의 토큰을 이 함수로 검사하고, 회원을 확인한 뒤 새 Access Token을 줍니다

쿠키의 토큰	결과	브라우저
정상 (3일 안)	`7`을 돌려주고 새 Access Token을 줍니다	로그인이 유지됩니다
만료, 위조, 종류가 `access`, 엉터리 문자열	401 "Please sign in again."	`refreshAccessToken()`이 `false`를 돌려주고 로그인 화면으로 갑니다

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt
```

```
from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다 (이번에 새로 추가한 것은 없습니다). **이 함수는 import한 것을 직접 쓰지 않습니다.** 검사는 같은 파일의 05번 `_decode_token()`이 하고, 그 함수가 `jwt`, `get_settings`, `AppError`를 사용합니다.

함수

```
def decode_refresh_token(token: str) -> int:
    return _decode_token(token, "refresh")
```

05번에서 검사 부분을 `_decode_token(token, expected_type)`으로 나눠 둔 덕분에 한 줄로 끝납니다. 05번 문서 §5에 "P1-05의 `decode_refresh_token()`은 `return _decode_token(token, "refresh")` 한 줄이 된다"고 적어 둔 그대로입니다.

문법 풀이

import 문법	뜻
(없음)	같은 파일의 함수는 import 없이 부릅니다

`_decode_token(token, "refresh")`가 하는 검사는 다음과 같습니다 (05번과 같음).

순서	검사	실패하면
①	서명, 만료, 필수 칸(`exp`, `sub`, `type`)을 확인하고, 알고리즘은 `HS256`만 허용합니다	401
②	`type`이 `"refresh"`인지 확인합니다	401
③	`sub`를 정수로 바꿉니다	401

4. 흐름

1. 05번의 `_decode_token()`에 종류 `"refresh"`를 넘깁니다.
2. 통과하면 회원 번호를 돌려주고, 아니면 401로 응답합니다.

5. 왜 이렇게

선택	이유
05번과 같은 검사 함수를 씀	서명, 만료, 알고리즘 고정(`alg: none` 차단) 규칙이 한곳에만 있게 됩니다. 05번을 고치면 이 함수도 함께 바뀝니다
Access Token을 거절함	30분짜리 토큰으로 재발급을 받을 수 있다면, 훔친 Access Token으로 새 토큰을 계속 받아서 30분 제한을 무력화할 수 있습니다
실패 이유와 상관없이 같은 401	05번과 같은 이유입니다. 브라우저는 어떤 실패든 "다시 로그인" 하나로 처리합니다
회원 상태(탈퇴, 정지)는 여기서 보지 않음	이 함수는 토큰만 봅니다. 회원 확인은 33번 라우터가 30번 `get_active_user()`로 합니다 (06번 `get_current_user`와 같은 역할 분담)

6. 시험 (`tests/test_security.py`에 8개를 추가해서 모두 25개)

시험	확인한 것
만들고 읽기	`decode_refresh_token(create_refresh_token(7)) == 7`입니다 (27번과 짝)
Access Token을 넣음	401입니다. 26번으로 만든 토큰은 거절합니다
이상한 토큰 6개	만료, 다른 비밀값, `sub: "admin"`, `exp` 없음, 엉터리 문자열, 빈 문자열은 모두 401 "Please sign in again."입니다

전체 시험 202개가 통과했고, `ruff` 검사도 통과했습니다.